

DRL-BASED ADAPTIVE NETWORK ROUTING

Complete Implementation Guide
From Environment Setup to Final Report

Deep Q-Network (DQN) + DDQN in ns-3 via ns3-gym
Inspired by DRSIR — IEEE Transactions on Network & Service Management 2021

Muhammad Sabeeh (23K-0002) | Rayyan Merchant (23K-0073)
Computer Networks — Semester Project | FAST NUCES

PyTorch • ns-3 3.38 • ns3-gym • NetworkX • Python 3.10+

STACK	TOPOLOGY	AGENT	BASELINE	SCENARIOS
ns-3 3.38 ns3-gym PyTorch	6 nodes / 10 links P2P mesh	Centralized DQN k=3 paths 1 hidden layer 50n	Dijkstra shortest-path static routing	Normal / Congested Link failure Mixed load

Table of Contents

1. Project Overview & Design Decisions	3
2. Topology Design	5
3. Environment Setup	7
4. Phase 1 — ns-3 Baseline Simulation	10
5. Phase 2 — Metric Validation	14
6. Phase 3 — ns3-gym Integration & Reward	16
7. Phase 4 — DQN Agent Implementation	21
8. Phase 5 — Training & Health Checks	28
9. Phase 6 — DDQN Upgrade	31
10. Evaluation & Experiments (4 Scenarios)	33
11. Plotting & Visualization	37
12. Report Writing Guide	40
13. Troubleshooting Reference	43
14. Final Project Checklist	46

SECTION 1 — PROJECT OVERVIEW & DESIGN DECISIONS

1. Project Overview & Design Decisions

This project implements a Deep Reinforcement Learning (DRL) adaptive routing system in ns-3. A single centralized DQN agent — directly inspired by DRSIR (Casas-Velasco et al., IEEE TNSM 2021) — learns to select optimal paths through a 6-node simulated network, outperforming static Dijkstra routing under congestion and link failures. The agent is then upgraded to DDQN for improved Q-value estimation.

1.1 Architecture Overview

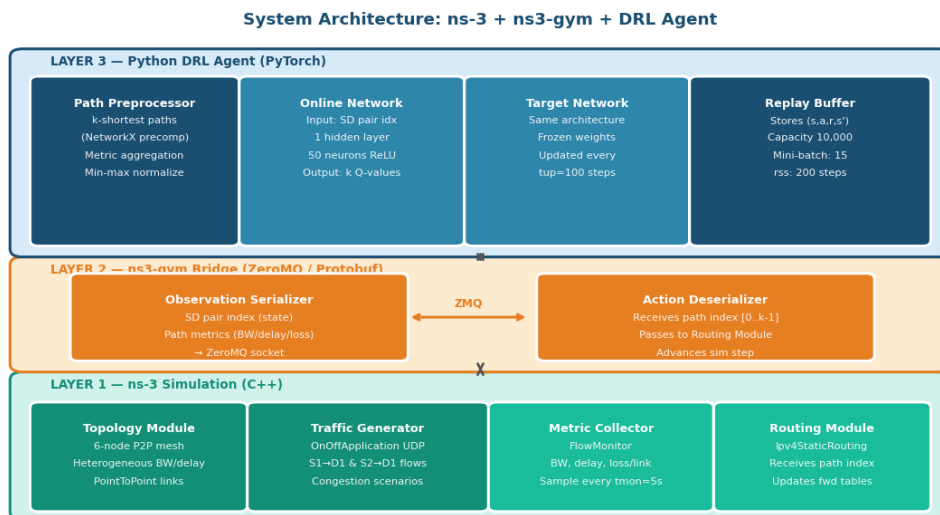


Figure 1: Three-layer system architecture. Each layer is independently testable. All data flows through the ZeroMQ bridge at Layer 2.

1.2 Locked Design Decisions

These decisions are final. Do not change them mid-project.

Decision	Choice	Rationale
Agent topology	Single centralized agent	Eliminates non-stationarity; matches DRSIR; one training loop, clean reward signal
Action space	Path selection (k=3 candidate paths)	Smaller action space than next-hop; encodes end-to-end quality; directly from DRSIR
NN input (state)	SD pair integer index only	Path-state metrics go to reward function ONLY — not NN input. Matches DRSIR
Framework	PyTorch (not TensorFlow)	Update proposal document. Better debugging, consistent with research tooling
Route installation	Ipv4StaticRoutingHelper in ns-3	ns-3 equivalent of DRSIR OpenFlow installation; simple, robust, well-documented
Reward direction	Agent MINIMIZES reward (cost)	Reward = cost function (higher BW = lower cost). argmin not argmax in action selection

1.3 MDP Formulation

MDP Element	Definition	Concrete Example
-------------	------------	------------------

State S	Integer: SD pair being routed	0 = S1→D1, 1 = S2→D1
Action A	Integer in [0,k-1]: which precomputed path to use	Action 0: S1-R1-D1 Action 1: S1-R2-D1 Action 2: S1-R1-R2-D1
Reward R	$R=B1*(1/bw_n)+B2*del_n+B3*loss_n$ (minimised)	High congestion path → high cost → agent avoids it
Transition	Metrics change after action applied & tmon=5s	Step 1, congestion changes R2-D1 delay; agent sees new state
Episode	Full 100s simulation; 20 decision steps (one per 5s)	100 Episodes total training; metrics logged per episode

2. Topology Design

The 6-node topology is intentionally heterogeneous. If all links had identical capacity and delay, Dijkstra and DQN would always choose the same path and comparison would be meaningless. The design creates scenarios where the shortest path by hop-count is NOT optimal under congestion — which is exactly what the DQN agent must learn to exploit.

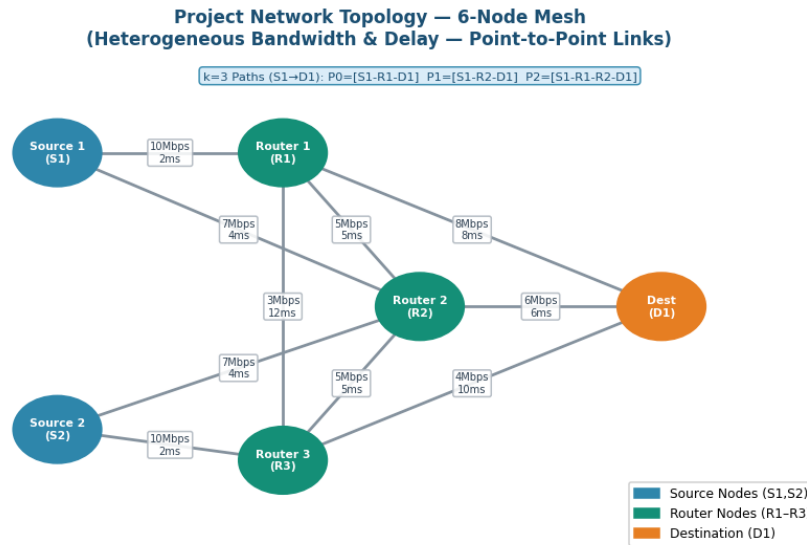


Figure 2: 6-node network topology. Blue=sources, Green=routers, Orange=destination. Link labels show bandwidth and propagation delay.

2.1 Node Definitions

Node	Role	Connected To	ns-3 Application
S1	Primary source (traffic gen)	R1, R2	OnOffApplication UDP 8 Mbps
S2	Secondary source	R2, R3	OnOffApplication UDP 6 Mbps
R1	Interior router	S1, R2, R3, D1	None (routing only)
R2	Interior hub (central)	S1, S2, R1, R3, D1	None (routing only)
R3	Interior router	S2, R1, R2, D1	None (routing only)
D1	Destination sink	R1, R2, R3	PacketSink (all flows)

2.2 Link Specifications

Link	BW	Delay	Strategic Role
S1 — R1	10 Mbps	2 ms	Fast entry to R1 path
S1 — R2	7 Mbps	4 ms	Medium entry via central hub R2

S2 — R2	7 Mbps	4 ms	S2 to central hub
S2 — R3	10 Mbps	2 ms	S2 fast entry via R3
R1 — R2	5 Mbps	5 ms	Interior bottleneck — congests under load
R2 — R3	5 Mbps	5 ms	Interior bottleneck — congests under load
R1 — D1	8 Mbps	8 ms	Fast direct exit — Dijkstra prefers this
R2 — D1	6 Mbps	6 ms	Medium exit — good alternative under R1 failure
R3 — D1	4 Mbps	10 ms	Slow exit — used only when others fail/congest
R1 — R3	3 Mbps	12 ms	Low-cap cross-link — rarely optimal

2.3 Candidate Paths (k=3, Precomputed by NetworkX)

ID	SD Pair	Route	Normal Rank	Design Purpose
P0	S1→D1	S1 - R1 - D1	Lowest (fast)	Direct path Dijkstra always picks
P1	S1→D1	S1 - R2 - D1	Medium	Reroute when R1 or R1-D1 congested/fails
P2	S1→D1	S1 - R1 - R2 - D1	Highest (3-hop)	Last resort path
P3	S2→D1	S2 - R3 - D1	Low	Direct S2 path, limited by slow R3-D1
P4	S2→D1	S2 - R2 - D1	Medium	Better exit capacity via R2
P5	S2→D1	S2 - R2 - R1 - D1	Highest	Alternate when R2-D1 congested

[KEY INSIGHT]

Dijkstra always picks P0 (S1→D1) and P3 (S2→D1) since it uses static link weights.

The DQN agent must learn that under congestion P1 outperforms P0 — this is your result.

This heterogeneity is what makes the comparison scientifically meaningful.

3. Environment Setup

3.1 System Requirements

Component	Requirement	Notes
OS	Ubuntu 20.04 or 22.04 LTS	Native Linux — NOT WSL2 for ns-3
RAM	8 GB min, 16 GB recommended	ns-3 + training can use 4-6 GB simultaneously
Python	3.10 or 3.11	Must match across venv and ns3-gym package
GCC	9.0 or higher	Required for ns-3 3.38 compilation
CMake	3.16 or higher	Required for ns-3 build system
Disk	10 GB free minimum	ns-3 build artifacts are large (~3 GB)

3.2 ns-3 3.38 Installation

```
# Step 1: Install system dependencies
sudo apt-get update
sudo apt-get install -y build-essential cmake git python3-dev python3-pip python3-venv
sudo apt-get install -y libzmq3-dev libprotobuf-dev protobuf-compiler pkg-config

# Step 2: Clone ns-3 3.38
cd ~
git clone https://gitlab.com/nsnam/ns-3-dev.git ns-3.38
cd ns-3.38 && git checkout ns-3.38

# Step 3: Configure and build (takes 10-20 min first time)
./ns3 configure --enable-examples --enable-tests
./ns3 build

# Step 4: Verify
./ns3 run hello-simulator
# Expected output: Hello Simulator
```

3.3 ns3-gym Installation

```
# Step 1: Clone ns3-gym into contrib/opengym (name is mandatory)
cd ~/ns-3.38
git clone https://github.com/tnk-tub/ns3-gym.git contrib/opengym

# Step 2: Rebuild ns-3 with the opengym module
./ns3 configure --enable-examples && ./ns3 build

# Step 3: Install Python package
pip3 install ns3gym

# Step 4: Verify bridge works (MUST pass before any project code)
# Terminal 1:
cd ~/ns-3.38 && ./ns3 run opengym-simple

# Terminal 2:
cd ~/ns-3.38/contrib/opengym/examples/opengym && python3 test_opengym.py
# Expected: observations printed, actions sent, no crash
```

3.4 Python Environment

```
python3 -m venv ~/drl-routing-env
source ~/drl-routing-env/bin/activate
# Install all required packages
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
pip install numpy matplotlib networkx gymnasium ns3gym pandas seaborn
# Verify
python3 -c "import torch; print(torch.__version__)"
python3 -c "import networkx; print(networkx.__version__)"
```

3.5 Project Folder Structure

```
~/ns-3.38/scratch/drl_routing/
routing_sim.cc # Main ns-3 simulation + RL environment hooks
routing_env.cc/.h # Custom OpenGymEnv subclass
~/drl_project/
configs/hyperparams.py # ALL hyperparameters – single source of truth
configs/topology.py # Topology graph definition (Python side)
agent/network.py # QNetwork (PyTorch)
agent/replay_buffer.py # Experience replay
agent/dqn_agent.py # DQNAgent class
env/path_precompute.py # NetworkX k-shortest paths
env/metrics.py # Reward computation (DRSIR formula)
env/ns3_wrapper.py # Python-side Gym wrapper
training/train_dqn.py # DQN main training script
training/train_ddqn.py # DDQN training script
training/evaluate.py # Evaluation + comparison
baseline/parse_flowmon.py # FlowMonitor XML parser
results/logs/ # Training CSV logs
results/plots/ # Generated figures (PDF)
results/raw/ # ns-3 FlowMonitor XML output
results/checkpoints/ # Model .pt checkpoints
```

SECTION 4 — PHASE 1: ns-3 BASELINE SIMULATION

1 ns-3 Baseline — Build & Validate the Network

Build the 6-node topology in ns-3, run Dijkstra routing, generate traffic, simulate congestion and link failure, and collect all metrics via FlowMonitor. This produces your baseline numbers that DQN must improve upon.

[DONE WHEN]

Phase done when: simulation runs 100s cleanly, FlowMonitor XML is non-empty, throughput/delay/loss values are sensible, and you can plot them from CSV.

Do NOT proceed to Phase 2 until every metric is non-zero and realistic.

4.1 ns-3 Simulation Code (routing_sim.cc — key sections)

Create ~/ns-3.38/scratch/drl_routing/routing_sim.cc. The full structure is shown below with all critical sections.

```
#include <ns3/core-module.h>
#include <ns3/network-module.h>
#include <ns3/internet-module.h>
#include <ns3/point-to-point-module.h>
#include <ns3/applications-module.h>
#include <ns3/flow-monitor-module.h>
#include <ns3/ipv4-static-routing-helper.h>
#include <ns3/ipv4-global-routing-helper.h>
using namespace ns3;

int main(int argc, char *argv[]) {
    double simTime = 100.0;
    bool enableFailure = false;
    double failTime = 40.0;
    std::string outFile = "baseline.xml";
    CommandLine cmd;
    cmd.AddValue("simTime", "Sim time", simTime);
    cmd.AddValue("failure", "Enable link fail", enableFailure);
    cmd.AddValue("output", "Output XML", outFile);
    cmd.Parse(argc, argv);

    // === 6 Nodes: S1=0, S2=1, R1=2, R2=3, R3=4, D1=5 ===
    NodeContainer nodes; nodes.Create(6);

    // === Define 10 P2P links ===
    struct LinkDef { int u,v; const char *bw,*delay; };
    LinkDef lnks[] = {
        {0, 2, "10Mbps", "2ms"}, {0, 3, "7Mbps", "4ms"},
        {1, 3, "7Mbps", "4ms"}, {1, 4, "10Mbps", "2ms"},
        {2, 3, "5Mbps", "5ms"}, {3, 4, "5Mbps", "5ms"},
        {2, 5, "8Mbps", "8ms"}, {3, 5, "6Mbps", "6ms"},
        {4, 5, "4Mbps", "10ms"}, {2, 4, "3Mbps", "12ms"}
    };

    PointToPointHelper p2p;
```

```

NetDeviceContainer devs[10];
for(int i=0;i<10;i++){
p2p.SetDeviceAttribute("DataRate",StringValue(lnks[i].bw));
p2p.SetChannelAttribute("Delay",StringValue(lnks[i].delay));
devs[i]=p2p.Install(nodes.Get(lnks[i].u),nodes.Get(lnks[i].v));
}
// === Internet stack + IP addressing ===
InternetStackHelper inet; inet.Install(nodes);
Ipv4AddressHelper addr;
for(int i=0;i<10;i++){
std::ostringstream sub; sub<<"10."<<i<<"1.0";
addr.SetBase(sub.str().c_str(),"255.255.255.0");
addr.Assign(devs[i]);
}
Ipv4GlobalRoutingHelper::PopulateRoutingTables(); // Dijkstra
// === Get D1 IP dynamically (NEVER hardcode) ===
Ipv4Address d1ip = nodes.Get(5)->GetObject<Ipv4>()
->GetAddress(1,0).GetLocal();
// === Traffic: S1->D1 (8 Mbps UDP) ===
uint16_t port = 9;
PacketSinkHelper sink("ns3::UdpSocketFactory",
InetSocketAddress(Ipv4Address::GetAny(),port));
sink.Install(nodes.Get(5));
OnOffHelper onoff("ns3::UdpSocketFactory",
InetSocketAddress(d1ip,port));
onoff.SetAttribute("DataRate",DataRateValue(DataRate("8Mbps")));
onoff.SetAttribute("OnTime",
StringValue("ns3::ConstantRandomVariable[Constant=1]"));
onoff.SetAttribute("OffTime",
StringValue("ns3::ConstantRandomVariable[Constant=0]"));
onoff.Install(nodes.Get(0)).Start(Seconds(1.0)); // S1
// === S2->D1 (6 Mbps UDP) – same pattern ===
OnOffHelper onoff2("ns3::UdpSocketFactory",
InetSocketAddress(d1ip,port));
onoff2.SetAttribute("DataRate",DataRateValue(DataRate("6Mbps")));
onoff2.SetAttribute("OnTime",
StringValue("ns3::ConstantRandomVariable[Constant=1]"));
onoff2.SetAttribute("OffTime",
StringValue("ns3::ConstantRandomVariable[Constant=0]"));
onoff2.Install(nodes.Get(1)).Start(Seconds(1.0)); // S2
// === Optional link failure: bring down R1-D1 at t=40s ===
if(enableFailure){
Ptr<RateErrorModel> err = CreateObject<RateErrorModel>();
err->SetAttribute("ErrorRate",DoubleValue(1.0)); // drop all
Simulator::Schedule(Seconds(failTime),
[&]() { devs[6].Get(0)->SetAttribute("ReceiveErrorModel",
PointerValue(err)); }); // devs[6] = R1-D1
}
// === FlowMonitor ===
FlowMonitorHelper fmh;
Ptr<FlowMonitor> fm = fmh.InstallAll();
Simulator::Stop(Seconds(simTime));
Simulator::Run();

```

```
fm->SerializeToXmlFile(outFile,true,true);
Simulator::Destroy();
return 0;
}
```

[CRITICAL]

devs[6] = R1-D1 link (index 6 in the lnsks[] array above).

D1 IP must be fetched with GetObject<Ipv4>() AFTER addr.Assign() is called.

Never hardcode any IP address — always resolve it dynamically at runtime.

4.2 Build and Run Commands

```
# Build the simulation
cd ~/ns-3.38
./ns3 build

# Run - Normal traffic scenario
./ns3 run "drl_routing/routing_sim --simTime=100 --failure=false --output=normal.xml"

# Run - Link failure scenario
./ns3 run "drl_routing/routing_sim --simTime=100 --failure=true --output=failure.xml"

# Run - High congestion (edit DataRate to 12Mbps in source, rebuild)
./ns3 run "drl_routing/routing_sim --simTime=100 --failure=false --output=congested.xml"
```

4.3 FlowMonitor XML Parser (baseline/parse_flowmon.py)

```
import xml.etree.ElementTree as ET
import pandas as pd

def parse_flowmon(xml_file, duration=99.0):
    tree = ET.parse(xml_file)
    root = tree.getroot()
    rows = []
    for flow in root.iter("Flow"):
        tx = int(flow.get("txPackets", 0))
        rx = int(flow.get("rxPackets", 0))
        loss_ratio = (tx-rx)/tx if tx>0 else 0
        # delaySum is in nanoseconds (e.g. "12345678ns")
        ds = float(flow.get("delaySum", "0ns").replace("ns",""))
        avg_delay_ms = (ds/rx/1e6) if rx>0 else 0
        rxb = int(flow.get("rxBytes", 0))
        tput_mbps = (rxb*8)/(duration*1e6)
        rows.append({"flow": flow.get("flowId"),
                    "throughput_mbps": tput_mbps,
                    "avg_delay_ms": avg_delay_ms,
                    "loss_ratio": loss_ratio})
    return pd.DataFrame(rows)

if __name__ == "__main__":
    df = parse_flowmon("normal.xml")
    print(df.to_string(index=False))
    df.to_csv("results/logs/baseline_normal.csv", index=False)
```

SECTION 5 — PHASE 2: METRIC VALIDATION

2

Metric Validation — Verify Your Numbers Are Real

This phase is skipped by most teams and is the single biggest cause of wasted time later. Before touching RL code, confirm that your FlowMonitor metrics respond correctly to topology changes. A broken metric pipeline produces a trained agent that appears to work but produces scientifically invalid results.

5.1 Required Validation Tests

Test	What To Do	Expected Result	Fail Signal → Fix
BW sensitivity	Reduce S1 rate 8→2 Mbps, rerun	Throughput drops proportionally	Same throughput → FlowMonitor not tracking S1
Congestion	Add 3rd OnOff app from R1, rerun	Loss ratio rises, delay rises	No change → traffic not saturating links
Delay probe	Change R1-D1 delay 8ms→50ms, rerun	Avg_delay_ms increases ~42ms	No change → delay tag not being parsed
Link failure	Run with --failure=true	Throughput drops at t=40s, may recover	Stays zero → error model not applied
Zero load	Remove all OnOff apps, rerun	FlowMonitor has zero flows	Flows present → phantom traffic

[GATE CONDITION]

ALL 5 tests must pass before proceeding to Phase 3.

If any test fails, fix the ns-3 code before continuing — do not work around it.

Document your passing test results; they belong in your report as validation evidence.

SECTION 6 — PHASE 3: ns3-gym INTEGRATION & REWARD

3 ns3-gym Integration — Convert Simulation to RL Environment

This phase converts the static ns-3 simulation into a Gym-compatible environment. The C++ side defines the environment dynamics. The Python side wraps it with a standard interface. After this phase, you must be able to run random actions for 50 episodes without crashes and observe changing rewards.

6.1 Communication Protocol

Step	ns-3 (C++) Does	Python Agent Does
1. Reset	Restart simulation, reset state	env.reset() → receives obs vector
2. Observe	Compute path metrics, serialize	Receives obs[0..9] via ZeroMQ
3. Act	Wait for action on ZMQ socket	env.step(action) → sends path index
4. Apply	Call Ipv4StaticRouting update	ns-3 confirms action received
5. Advance	Run sim for tmon=5 simulated seconds	Waits for next observation
6. Reward	Compute new metrics, send obs+done	Receives (obs, _, done, info)
7. Repeat	Until t >= simTime (100s)	Compute reward, store experience, train

6.2 Observation Space Design

The observation vector has 10 elements. The neural network uses ONLY element 0 (SD pair index). Elements 1–9 are used ONLY for reward computation. This separation is critical and matches DRSIR exactly.

```
# Observation vector – 10 floats total
# obs[0] : SD pair index (0 = S1->D1, 1 = S2->D1)
# obs[1:4] : Path 0 metrics [bw_raw, delay_raw, loss_raw]
# obs[4:7] : Path 1 metrics [bw_raw, delay_raw, loss_raw]
# obs[7:10]: Path 2 metrics [bw_raw, delay_raw, loss_raw]
# In Python (env/ns3_wrapper.py):
import gymnasium as gym
import numpy as np
observation_space = gym.spaces.Box(
    low=0.0, high=1.0, shape=(10,), dtype=np.float32
)
action_space = gym.spaces.Discrete(3) # k=3 candidate paths
# CRITICAL: obs[0] goes to NN input as int(obs[0])
# obs[1:] goes to reward computation – NOT to NN
```

6.3 Path Precomputation (env/path_precompute.py)

```
import networkx as nx
def build_graph():
```

```

G = nx.Graph()
# Nodes: 0=S1, 1=S2, 2=R1, 3=R2, 4=R3, 5=D1
G.add_edges_from([
(0,2,{"bw":10,"delay":2}), (0,3,{"bw":7,"delay":4}),
(1,3,{"bw":7,"delay":4}), (1,4,{"bw":10,"delay":2}),
(2,3,{"bw":5,"delay":5}), (3,4,{"bw":5,"delay":5}),
(2,5,{"bw":8,"delay":8}), (3,5,{"bw":6,"delay":6}),
(4,5,{"bw":4,"delay":10}), (2,4,{"bw":3,"delay":12}),
])
return G

def k_paths(G, src, dst, k=3):
gen = nx.shortest_simple_paths(G, src, dst, weight="delay")
return [next(gen) for _ in range(k)]

# Run once at startup – results are static (topology never changes)
G = build_graph()
PATHS = {
(0,5): k_paths(G,0,5), # S1->D1: [[0,2,5],[0,3,5],[0,2,3,5]]
(1,5): k_paths(G,1,5), # S2->D1: [[1,4,5],[1,3,5],[1,3,2,5]]
}

# Action 0 for SD(0,5) = path [0,2,5] = S1-R1-D1
# Action 1 for SD(0,5) = path [0,3,5] = S1-R2-D1

```

6.4 DRSIR Reward Formula

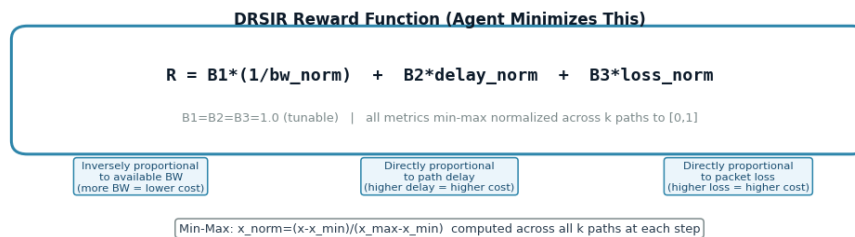


Figure 3: DRSIR reward formula. The agent minimizes this cost. All three metrics are min-max normalized across k paths at each step.

6.5 Reward Implementation (env/metrics.py)

```

import numpy as np

def compute_reward(obs, action, k=3, beta=(1.0,1.0,1.0)):
    """
    Compute DRSIR cost for chosen action. Agent MINIMIZES this.
    obs: full 10-element observation vector
    action: integer in [0, k-1]
    """
    # Extract path metrics: shape (k, 3) – columns: [bw, delay, loss]
    path_metrics = obs[1:].reshape(k, 3).astype(np.float32)
    # Min-max normalize each column across k paths
    norm = np.zeros_like(path_metrics)
    for col in range(3):
        vals = path_metrics[:, col]
        vmin, vmax = vals.min(), vals.max()
        if (vmax - vmin) < 1e-8:
            norm[:, col] = 0.5 # all paths equal for this metric
    
```

```

else:
norm[:, col] = (vals - vmin) / (vmax - vmin)
bw_n, dl_n, ls_n = norm[action]
# DRSIR Equation 5: inversely proportional to BW, directly to delay+loss
reward = beta[0]*(1.0/(bw_n+1e-6)) + beta[1]*dl_n + beta[2]*ls_n
return float(reward)
# Sanity check:
# High BW, low delay, low loss -> bw_n=1, dl_n=0, ls_n=0 -> R = 1/(1+eps) ~ 1.0 (low cost)
# Low BW, high delay, high loss -> bw_n=0, dl_n=1, ls_n=1 -> R = 1/eps + 1 + 1 >> 1 (high cost)

```

[GATE CONDITION]

Phase 3 done when: 50-episode random action loop runs without crash,
rewards differ across actions and steps, and ns-3 routing table changes per action.
Verify routing change with: NS_LOG="Ipv4StaticRouting" ./ns3 run ...

SECTION 7 — PHASE 4: DQN AGENT IMPLEMENTATION

4

DQN Agent — Network + Replay Buffer + Training Loop

7.1 Hyperparameter Table (source: DRSIR Table I)

Use these values exactly. Do NOT tune until baseline DQN converges.

Parameter	Value	DRSIR Source	Rationale
Hidden layers	1	Table I	Single layer proved sufficient; fastest convergence
Neurons	50	Table I	Best reward/speed tradeoff in DRSIR experiments
Discount factor γ	0.1	Table I	Near-sighted; routing decisions have immediate effect
Replay start (rss)	200 steps	Table I ÷2	Smaller topology needs less warmup
ϵ max / ϵ min	1.0 / 0.05	Standard	Full exploration at start; 5% minimum
Decay rate	1/400	Table I	Moderately fast transition to exploitation
Batch size	15	Table I	Small batches converged faster than 30 or 45
Target update freq	100 steps	Table I	Sync Target NN every 100 training steps
Buffer size	10,000	Standard	Covers 500 episodes × 20 steps with margin
Learning rate	0.001	Adam default	Adam optimizer with standard lr
k (paths)	3	Project	Reduced from DRSIR k=20 for 6-node topology
tmon	5 sim-seconds	Project	Decision interval; 20 steps per episode
Episodes	500	Project	Sufficient for convergence; monitor from ep 100

7.2 configs/hyperparams.py

```
# configs/hyperparams.py – Single source of truth. Import from everywhere.
HIDDEN_NEURONS = 50
GAMMA = 0.1
REPLAY_START = 200
EPS_MAX = 1.0
EPS_MIN = 0.05
DECAY_RATE = 1.0 / 400.0
BATCH_SIZE = 15
TARGET_UPDATE = 100
BUFFER_SIZE = 10_000
LR = 0.001
K_PATHS = 3
T_MON = 5.0 # simulation seconds per step
SIM_TIME = 100.0 # total simulation time per episode
N_EPISODES = 500
STEPS_PER_EP = int(SIM_TIME / T_MON) # = 20
```

```
BETA = (1.0, 1.0, 1.0) # reward weights
OBS_SIZE = 1 + K_PATHS * 3 # = 10
N_SD_PAIRS = 2 # S1->D1, S2->D1
```

7.3 agent/network.py — QNetwork

```
import torch, torch.nn as nn, torch.nn.functional as F
from configs.hyperparams import HIDDEN_NEURONS, K_PATHS, N_SD_PAIRS

class QNetwork(nn.Module):
    """Online and Target network – identical architecture."""
    def __init__(self):
        super().__init__()
        # Embed SD pair index into hidden space
        self.embed = nn.Embedding(N_SD_PAIRS, HIDDEN_NEURONS)
        self.fc1 = nn.Linear(HIDDEN_NEURONS, HIDDEN_NEURONS)
        self.out = nn.Linear(HIDDEN_NEURONS, K_PATHS)
        # Glorot (Xavier) uniform initialisation – from DRSIR
        nn.init.xavier_uniform_(self.fc1.weight)
        nn.init.xavier_uniform_(self.out.weight)

    def forward(self, idx):
        """idx: LongTensor shape (batch,) → Q-values shape (batch, k)"""
        x = self.embed(idx) # (batch, hidden)
        x = F.relu(self.fc1(x)) # (batch, hidden)
        return self.out(x) # (batch, k)

    def best_action(self, idx, device):
        """Greedy action – argmin because we MINIMISE cost."""
        with torch.no_grad():
            t = torch.tensor([idx], dtype=torch.long).to(device)
            return self.forward(t).argmin().item()
```

7.4 agent/replay_buffer.py

```
from collections import deque
import random, numpy as np

class ReplayBuffer:
    def __init__(self, capacity):
        self.buf = deque(maxlen=capacity)

    def push(self, s, a, r, ns, done):
        self.buf.append((s, a, r, ns, done))

    def sample(self, n):
        batch = random.sample(self.buf, n)
        s,a,r,ns,d = zip(*batch)
        return (np.array(s), np.array(a),
                np.array(r,dtype=np.float32),
                np.array(ns), np.array(d,dtype=np.float32))

    def __len__(self): return len(self.buf)
```

7.5 agent/dqn_agent.py — Full DQN Agent

```
import torch, torch.nn as nn, torch.optim as optim
import numpy as np, random
from agent.network import QNetwork
from agent.replay_buffer import ReplayBuffer
```

```

from configs.hyperparams import *

class DQNAgent:
def __init__(self, device="cpu"):
self.device = device
self.online = QNetwork().to(device)
self.target = QNetwork().to(device)
self.target.load_state_dict(self.online.state_dict())
self.target.eval()
self.optim = optim.Adam(self.online.parameters(), lr=LR)
self.buffer = ReplayBuffer(BUFFER_SIZE)
self.steps = 0
self.epsilon = EPS_MAX
self.losses = []
def act(self, state_idx):
"""Epsilon-greedy action selection."""
self.epsilon = max(EPS_MIN, EPS_MAX - self.steps * DECAY_RATE)
if random.random() < self.epsilon:
return random.randint(0, K_PATHS-1) # explore
return self.online.best_action(state_idx, self.device) # exploit
def store(self, s, a, r, ns, done):
self.buffer.push(s, a, r, ns, done)
self.steps += 1
def train_step(self):
if len(self.buffer) < REPLAY_START: return None
s,a,r,ns,d = self.buffer.sample(BATCH_SIZE)
S = torch.LongTensor(s).to(self.device)
A = torch.LongTensor(a).to(self.device)
R = torch.FloatTensor(r).to(self.device)
NS = torch.LongTensor(ns).to(self.device)
D = torch.FloatTensor(d).to(self.device)

# Current Q-values from Online Net
q_pred = self.online(S).gather(1, A.unsqueeze(1)).squeeze(1)
# DQN target: R + gamma * min_a Q_target(s_next, a)
with torch.no_grad():
q_next = self.target(NS).min(1)[0] # min because we MINIMISE
q_target = R + GAMMA * q_next * (1 - D)
loss = nn.MSELoss()(q_pred, q_target)
self.optim.zero_grad(); loss.backward(); self.optim.step()
self.losses.append(loss.item())
if self.steps % TARGET_UPDATE == 0:
self.target.load_state_dict(self.online.state_dict())
return loss.item()
def save(self, path):
torch.save(self.online.state_dict(), path)
def load(self, path):
self.online.load_state_dict(torch.load(path))
self.target.load_state_dict(self.online.state_dict())

```

7.6 DQN Training Loop Diagram

DQN Training Loop — Decision Flowchart

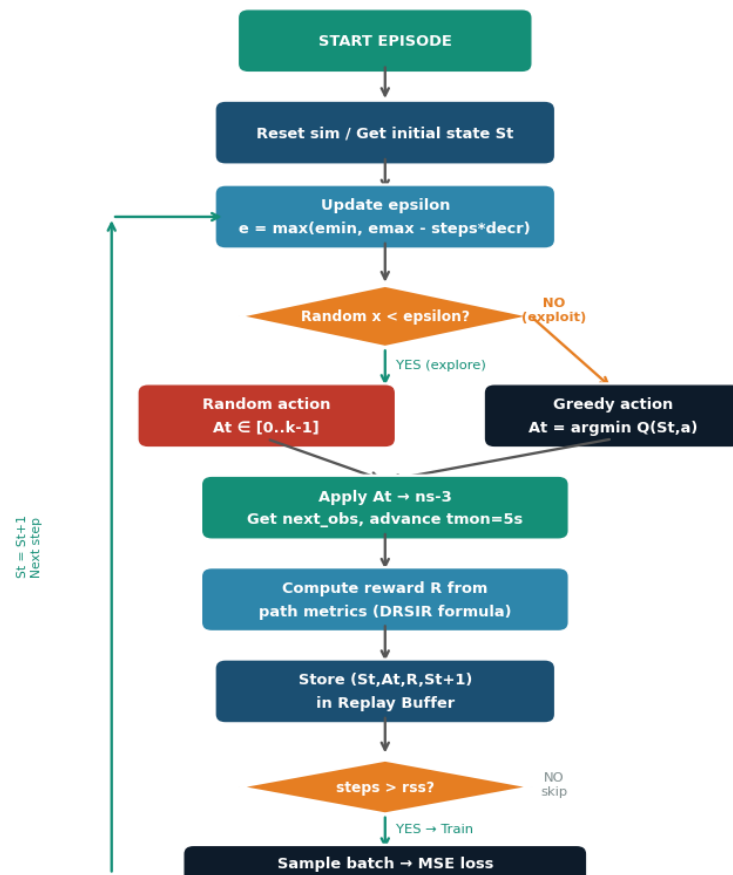


Figure 4: Complete DQN decision flowchart. Note argmin (not argmax) — agent minimizes cost.

5 Training the DQN Agent — Main Script & Health Monitoring

8.1 training/train_dqn.py — Main Training Script

```
import numpy as np, csv, os
from agent.dqn_agent import DQNAgent
from env.ns3_wrapper import NS3RoutingEnv
from env.metrics import compute_reward
from configs.hyperparams import *

def train():
    env = NS3RoutingEnv(port=5555)
    agent = DQNAgent(device="cpu")
    log = []
    os.makedirs("results/checkpoints", exist_ok=True)
    for ep in range(N_EPISODES):
        obs = env.reset()
        ep_cost = 0.0
        ep_losses = []
        for step in range(STEPS_PER_EP):
            state = int(obs[0]) # SD pair index -> NN input
            action = agent.act(state)
            next_obs, _, done, _ = env.step(action)
            reward = compute_reward(obs, action, K_PATHS, BETA)
            next_state = int(next_obs[0])
            agent.store(state, action, reward, next_state, float(done))
            loss = agent.train_step()
            if loss is not None: ep_losses.append(loss)
            ep_cost += reward
            obs = next_obs
            if done: break
        avg_loss = float(np.mean(ep_losses)) if ep_losses else 0.0
        log.append({"episode":ep,"reward":ep_cost,
            "avg_loss":avg_loss,"epsilon":agent.epsilon})
        if ep % 50 == 0:
            print(f"Ep {ep:4d} | Cost {ep_cost:.4f} | "
                f"Loss {avg_loss:.6f} | Eps {agent.epsilon:.3f}")
            agent.save(f"results/checkpoints/dqn_ep{ep}.pt")
            agent.save("results/checkpoints/dqn_final.pt")
        keys = log[0].keys()
        with open("results/logs/dqn_training.csv","w",newline="") as f:
            w = csv.DictWriter(f, fieldnames=keys)
            w.writeheader(); w.writerows(log)
        print("Training complete.")
    if __name__ == "__main__":
        train()
```

8.2 Training Health Checks

Signal	Healthy	Problem → Likely Cause
Episode cost	Decreasing trend over 500 eps	Flat or increasing → reward formula inverted or LR too high
Training loss	High then gradually decreasing	NaN → LR too high. Always zero → buffer not filling
Epsilon	1.0 → 0.05 by episode ~400	Stays 1.0 → steps counter broken; drops immediately → decay too fast
Action distribution	Uniform early, concentrated late	Always same action from ep 1 → exploration not working
Reward variance	High early (exploring), lower late	Always zero variance → ns-3 returning same obs every step

SECTION 9 — PHASE 6: DDQN UPGRADE

6

DDQN Upgrade — One Formula Change, Full Comparison

DDQN fixes DQN's overestimation bias. Since we minimize cost, overestimation means the agent thinks a bad path is better than it is. The fix separates action selection (Online NN) from action evaluation (Target NN). Everything else stays identical — same architecture, same buffer, same hyperparams.

9.1 DQN vs DDQN Target Formula

	DQN (current)	DDQN (upgrade)
Target Q	$R + \gamma \cdot \min_a Q_{\text{target}}(s', a)$	$R + \gamma \cdot Q_{\text{target}}(s', \operatorname{argmin}_a Q_{\text{online}}(s', a))$
Who selects action	Target NN (same net selects AND evaluates)	Online NN selects; Target NN evaluates only
Python code change	<code>q_next.min(1)[0]</code>	<code>best=online(NS).argmin(1,keepdim=True)</code> <code>q_next=target(NS).gather(1,best).squeeze(1)</code>
Everything else	Unchanged	Identical network, buffer, epsilon, hyperparams

9.2 DDQNAgent (subclass — train_step only)

```
# agent/dqn_agent.py — add below DQNAgent class
class DDQNAgent(DQNAgent):
    """DDQN: overrides only train_step(). Everything else inherited."""
    def train_step(self):
        if len(self.buffer) < REPLAY_START: return None
        s,a,r,ns,d = self.buffer.sample(BATCH_SIZE)
        S = torch.LongTensor(s).to(self.device)
        A = torch.LongTensor(a).to(self.device)
        R = torch.FloatTensor(r).to(self.device)
        NS = torch.LongTensor(ns).to(self.device)
        D = torch.FloatTensor(d).to(self.device)
        q_pred = self.online(S).gather(1, A.unsqueeze(1)).squeeze(1)
        with torch.no_grad():
            # DDQN: Online NN selects the best action
            best = self.online(NS).argmin(1, keepdim=True)
            # DDQN: Target NN evaluates ONLY that selected action
            q_next = self.target(NS).gather(1, best).squeeze(1)
            q_target = R + GAMMA * q_next * (1 - D)
        loss = nn.MSELoss()(q_pred, q_target)
        self.optim.zero_grad(); loss.backward(); self.optim.step()
        self.losses.append(loss.item())
        if self.steps % TARGET_UPDATE == 0:
            self.target.load_state_dict(self.online.state_dict())
        return loss.item()

# training/train_ddqn.py: identical to train_dqn.py
# Replace: agent = DQNAgent() → agent = DDQNAgent()
# Save logs to: results/logs/ddqn_training.csv
```

10. Evaluation & Experiments

Run three algorithms (Dijkstra, DQN, DDQN) across four scenarios. Fix all scenario parameters before running any experiments so all comparisons are made under identical conditions.

10.1 Evaluation Scenarios

Scenario 1: Normal Traffic

- Traffic: S1→D1 @ 4 Mbps, S2→D1 @ 3 Mbps — below all link capacities
- Duration: 100s simulation, 3 runs per algorithm, average results
- Purpose: establish baseline reference numbers for comparison
- Expected: Dijkstra and DQN similar performance; DQN slightly better

Scenario 2: High Congestion

- Traffic: S1→D1 @ 9 Mbps, S2→D1 @ 8 Mbps — saturates R1-R2 and R2-R3 bottlenecks
- Duration: 100s, 3 runs per algorithm
- Purpose: primary demonstration of DRL advantage
- Expected: DQN/DDQN 20-40% lower loss and delay than Dijkstra

Scenario 3: Link Failure

- Traffic: normal load. Link R1-D1 (devs[6]) fails at t=40s via RateErrorModel
- Duration: 100s — failure window t=40..100s
- Purpose: test dynamic rerouting under topology change
- Expected: Dijkstra drops packets for ~30s; DQN reroutes within tmon=5s

Scenario 4: Mixed/Dynamic Load

- Traffic: OnOff pattern — high load t=0-30s, low t=30-60s, high t=60-100s
- Duration: 100s, measure rolling 10s average throughput
- Purpose: test continuous adaptation to traffic variation
- Expected: DQN tracks optimal path across load changes; Dijkstra cannot

10.2 Evaluation Protocol

```

# training/evaluate.py
# Run AFTER all three models are trained
import subprocess, pandas as pd
from baseline.parse_flowmon import parse_flowmon
SCENARIOS = ["normal", "congested", "failure", "mixed"]
ALGOS = ["dijkstra", "dqn", "ddqn"]
RUNS = 3 # average across 3 runs per cell
def run_ns3(algo, scenario, run_id):
    """Launch ns-3 with the given routing policy."""
    out = f"results/raw/{algo}_{scenario}_run{run_id}.xml"
    cmd = [
        "./ns3", "run",
        f"drl_routing/routing_sim"
        f" --algo={algo} --scenario={scenario} --output={out}"
    ]
    subprocess.run(cmd, cwd="~/ns-3.38", check=True)
    return out
results = {}
for sc in SCENARIOS:
    results[sc] = {}
    for algo in ALGOS:
        runs_data = []
        for r in range(RUNS):
            xml = run_ns3(algo, sc, r)
            df = parse_flowmon(xml)
            runs_data.append({
                "throughput": df["throughput_mbps"].mean(),
                "delay": df["avg_delay_ms"].mean(),
                "loss": df["loss_ratio"].mean(),
            })
        avg = pd.DataFrame(runs_data).mean().to_dict()
        results[sc][algo] = avg
# Save all results
for sc, sc_data in results.items():
    pd.DataFrame(sc_data).T.round(4).to_csv(
        f"results/logs/{sc}_comparison.csv")
    print(f"\n=== {sc.upper()} ===")
    print(pd.DataFrame(sc_data).T.round(4).to_string())

```

[SCIENTIFIC RIGOR]

Run 3 repetitions per cell and average — this removes lucky/unlucky seeds.

Fix the random seed in ns-3: `RngSeedManager::SetSeed(42)` for reproducibility.

Results may show DQN not always beating Dijkstra — that is scientifically valid.

Honest discussion of results (even negative ones) is better than inflated claims.

11. Plotting & Visualization

Generate exactly 6 figures. All plots from saved CSV files — never from in-memory data — so they are fully reproducible.

Figure	Title & Purpose	Source File	Key Requirement
Fig 1	Training Convergence Episode cost vs episode (DQN & DDQN)	results/logs/dqn_training.csv results/logs/ddqn_training.csv	Use 20-ep rolling avg. Y-axis: cost (lower=better). Both curves on same plot.
Fig 2	Throughput Comparison Grouped bar across 4 scenarios	results/logs/*_comparison.csv	3 groups (Dijkstra/DQN/DDQN), 4 scenario clusters. Error bars from 3 runs.
Fig 3	Delay Comparison Grouped bar across 4 scenarios	results/logs/*_comparison.csv	Same layout as Fig 2. Y-axis: avg delay (ms). Annotate congestion scenarios.
Fig 4	Packet Loss Comparison Grouped bar across 4 scenarios	results/logs/*_comparison.csv	Same layout. Y-axis: loss ratio [0,1]. Congestion scenario shows bigger loss.
Fig 5	Failure Recovery Time-series throughput during failure (per scenario)	results/raw/failure_*.xml (parse per 10s window)	X: time (s), Y: throughput. Mark t=40s failure event with vertical line.
Fig 6	Training Loss Loss vs training steps (DQN & DDQN)	results/logs/dqn_training.csv results/logs/ddqn_training.csv	X: steps, Y: loss (log scale). Shows convergence stability.

11.1 Plotting Code (results/plots/generate_all.py)

[illegible]

12. Report Writing Guide

A semester-level report demonstrates understanding of theory, justified design decisions, and reproducible results. Follow this exact section structure. Word counts are targets, not limits.

Abstract (150-200 words)

- 1. Problem: static routing fails under congestion and failures.
- 2. Approach: centralized DQN with DRSIR-inspired path-state reward in ns-3.
- 3. Results: DQN achieved X% lower loss and Y% lower delay vs Dijkstra under congestion.
- 4. Conclusion: DRL provides practical adaptive routing for dynamic conditions.

1. Introduction (400-500 words)

- 1.1 Problem — why Dijkstra fails: static metrics, no adaptation, congestion blindness.
- 1.2 Motivation — cite DRSIR (Casas-Velasco et al., IEEE TNSM 2021) and Sutton & Barto.
- 1.3 Contribution — what your project adds: ns-3 implementation, DRSIR-inspired adaptation.
- 1.4 Paper structure — one sentence per section.

2. Background & Related Work (500-600 words)

- 2.1 RL basics: MDP, Q-learning, DQN, DDQN — define mathematically.
- 2.2 DRSIR: path-state metrics, centralized DQN, reward formula — explain the paper.
- 2.3 Your adaptation: ns-3 vs Mininet, StaticRouting vs OpenFlow, differences justified.
- 2.4 Brief survey of 2-3 other DRL routing papers from DRSIR Table II.

3. System Design (600-700 words)

- 3.1 Topology: 6-node mesh with Figure 2, justify heterogeneous links.
- 3.2 MDP: state, action, reward (include exact formula Eq.5), episode definition.
- 3.3 Architecture: include Figure 1 (three-layer diagram), explain data flow.
- 3.4 DQN: 1 hidden layer, 50 neurons, Glorot init, Adam — justify from DRSIR Table I.
- 3.5 DDQN: explain double estimation bias, state the single formula change.

4. Implementation (400-500 words)

- 4.1 ns-3 setup: topology, traffic generation, FlowMonitor, metric extraction.
- 4.2 ns3-gym bridge: observation/action design, tmon=5s step interval.

- 4.3 Path precomputation: NetworkX k-shortest paths, how paths are indexed.
- 4.4 Training: hyperparameter table (Table 1 in your report) — cite DRSIR Table I.

5. Experiments & Results (700-900 words)

- 5.1 Setup: sim parameters, hardware, seeds, number of runs per scenario.
- 5.2 Training convergence: Figures 1 & 6 — describe the curves and what they show.
- 5.3 Scenario results: one paragraph per scenario referencing Figures 2-5.
- 5.4 DQN vs DDQN: quantify improvement — is DDQN consistently better and by how much?
- 5.5 Failure analysis: quantify recovery time gap between Dijkstra and DQN (Figure 5).

6. Discussion (300-400 words)

- 6.1 Strongest result and its implication.
- 6.2 Limitations: small topology, simulated traffic, ns3-gym latency overhead.
- 6.3 Comparison to DRSIR: did you observe similar trends? What differed and why?
- 6.4 Future work: larger topologies, actor-critic (PPO/DDPG), real hardware.

7. Conclusion (150-200 words)

- Summarize: what you built, how it performed, what it demonstrates.
- Restate key quantitative result: best metric improvement under worst scenario.
- Final claim: DRL-based adaptive routing is viable and practical for dynamic networks.

13. Troubleshooting Reference

Work through this list systematically before asking for help.

ns-3 build fails — CMake protobuf error

- `sudo apt-get install --reinstall libprotobuf-dev protobuf-compiler`
- `cd ~/ns-3.38 && ./ns3 clean && ./ns3 configure && ./ns3 build`
- Check GCC version: `gcc --version` (must be ≥ 9.0)

ns3-gym ZeroMQ connection refused

- Start ns-3 simulation BEFORE the Python script
- Check both use port 5555: `grep -r "5555" contrib/opengym/`
- Check firewall: `sudo ufw allow 5555`
- Verify listening: `netstat -tuln | grep 5555`

FlowMonitor XML is empty or has zero flows

- Check OnOff app installed on correct node: `nodes.Get(0)` for S1
- Verify PacketSink is on D1 with matching port number
- Enable logging: `LogComponentEnable("OnOffApplication", LOG_LEVEL_INFO)`
- Check D1 IP: print it with `std::cout << d1ip << std::endl`

DQN reward is constant every episode

- ns-3 is returning same metrics — check metric polling after sim advances
- Print raw obs vector: `std::cout << obs[0] << " " << obs[1] << std::endl`
- Inject traffic variation: use `RandomVariableStream` for OnOff DataRate
- Verify tmon loop is actually advancing simulation time

Training loss explodes to NaN

- Reduce LR from 0.001 to 0.0001
- Add gradient clipping: `nn.utils.clip_grad_norm_(agent.online.parameters(), 1.0)`
- Check rewards are bounded: print min/max reward per episode
- Check for NaN in input: `assert not torch.isnan(S).any()`

Agent always selects same action from episode 1

- Epsilon not decaying: `print(agent.epsilon)` at each step
- Verify steps counter increments: `print(agent.steps)` in `act()`
- Check `EPS_MAX=1.0` and `EPS_MIN=0.05` are imported correctly

ns-3 crashes during RL training loop

- Add `RngSeedManager::SetSeed(42)` for reproducible runs
- Check `Simulator::Destroy()` is called at episode end, not mid-episode
- Use Valgrind: `valgrind --error-exitcode=1 ./ns3 run ...`
- Check that `ns3-gym reset()` fully reinitializes the simulation state

SECTION 14 — FINAL PROJECT CHECKLIST

14. Final Project Checklist

Use this as your daily tracker. Do not proceed to the next phase until every item in the current phase is checked.

Phase 1 — Baseline

- ns-3 3.38 builds without errors
- ns3-gym hello-world runs in both terminals without crash
- 6-node simulation runs 100s cleanly
- FlowMonitor XML non-empty with sensible throughput, delay, loss
- Dijkstra baseline numbers saved to CSV and plotted

Phase 2 — Validation

- BW sensitivity test passes
- Congestion injection increases loss AND delay
- Link failure causes throughput drop
- Delay probe increases avg_delay_ms correctly
- Zero-load test returns zero flows

Phase 3 — Integration

- Random action loop: 50 episodes, zero crashes
- Observations change between steps
- Rewards are NOT constant across different actions
- ns-3 routing table changes when different action is applied

Phase 4 — DQN

- hyperparams.py is the single source of all constants
- QNetwork, ReplayBuffer, DQNAgent all implemented and unit-tested
- train_dqn.py runs end-to-end and saves training log CSV
- Episode cost shows downward trend over 500 episodes
- Model checkpoint saved at ep 0, 250, 500

Phase 5 — Training

- All 5 health check signals are healthy
- Epsilon decays from 1.0 to 0.05 by episode ~400
- Training log CSV contains episode, reward, avg_loss, epsilon

Phase 6 — DDQN

- DDQNAgent subclass implemented with correct DDQN target formula
- DDQN trains 500 episodes without breaking pipeline
- DDQN training log saved separately (ddqn_training.csv)

Evaluation

- All 4 scenarios run for all 3 algorithms (3 runs each)
- All 4 comparison CSVs saved to results/logs/
- All 6 figures generated as PDF with axis labels and legend

Report

- Abstract states quantitative results (not vague claims)
- Section 3 includes topology figure and exact reward formula
- Section 4 includes hyperparameter table citing DRSIR Table I
- Section 5 discusses each scenario with figure references
- Section 6 honestly discusses limitations
- References include DRSIR IEEE TNSM 2021 and Sutton & Barto 2018

Deep Reinforcement Learning-Based Adaptive Network Routing | Muhammad Sabeeh (23K-0002) & Rayyan Merchant (23K-0073) | FAST NUCES